

Mini Payment Gateway

Product Requirements Document (PRD)

Version: 1.0 **Status:** Draft **Product:** B2B Payment Infrastructure

Executive Summary

A developer-friendly payment gateway that allows merchants to integrate once, accept payments through Cray initially, generate hosted payment links, automatically credit a merchant wallet, track transaction and balance history, and initiate withdrawals. The architecture will abstract payment providers so additional providers can be added without changing the merchant-facing API.

1. Product Overview

The platform will provide a unified merchant experience for payment acceptance, hosted checkout, transaction management, wallet accounting and withdrawals. Cray will be the first payment provider, while the provider layer will be designed to support additional providers in the future.

2. Problem Statement

Businesses often need to integrate payment providers, maintain transaction records, handle callbacks and webhooks, build checkout pages, reconcile payments and manage settlement. This product abstracts those responsibilities behind one API and dashboard.

3. Product Goals

Goal	Description
Unified Payment API	One API for merchants regardless of the underlying provider.
Hosted Checkout	Generate payment links and customer-facing checkout pages.
Merchant Wallet	Maintain balances and a reliable financial ledger.
Provider Abstraction	Start with Cray and add providers later.
Merchant Dashboard	Transactions, balances, payment links, withdrawals and API keys.
Developer Experience	Simple authentication, idempotency, verification and webhooks.

4. User Types

Merchant: Business accepting payments through the platform.

Developer: Technical user integrating the gateway API and webhooks.

Customer: End user making a payment through a hosted checkout; no account required.

Platform Administrator: Internal operator managing merchants, providers, transactions and withdrawals.

5. Core MVP Features

5.1 Merchant Registration & Authentication

Registration should collect business name, email, phone, country, currency, business type and password. The platform should support email verification, login, logout, password reset, session management and rate limiting.

5.2 Merchant Dashboard

Dashboard overview should show available balance, pending balance, total payment volume, transaction count, withdrawals and recent activity. Navigation should include Overview, Transactions, Payment Links, Wallet, Withdrawals, API Keys, Webhooks and Settings.

5.3 API Key Management

Merchants can generate, name, rotate and revoke API keys. Separate sandbox/test and production environments should be supported. Secret keys should only be displayed at creation and stored securely.

5.4 Payment API

Merchants can initialize payments using amount, currency, customer information, merchant reference, callback URL and metadata. The response returns an internal payment ID, reference and hosted checkout URL.

Example endpoint: **POST /api/v1/payments**

6. Payment Lifecycle

Recommended internal states: INITIALIZED → PENDING → PROCESSING → SUCCESS, with alternative terminal states FAILED, CANCELLED, EXPIRED and REFUNDED.

Provider status must be normalized into the gateway's own payment status. A redirect or provider callback alone should not be sufficient to credit a merchant wallet; the gateway should independently verify the payment.

7. Provider Integration Architecture

The platform should use a provider adapter/interface. A generic PaymentProvider contract should expose operations such as initializePayment(), verifyPayment(), processWebhook(), getPaymentStatus(), refundPayment() and createPayout().

Architecture: Merchant API → Payment Service → Provider Interface → Cray / Future Providers.

Cray is the initial provider. Its exact API endpoints, authentication, webhook payloads, supported payment methods, currencies and payout capabilities must be confirmed against current Cray technical documentation during implementation.

8. Payment Links & Hosted Checkout

Merchants can create payment links by specifying amount, currency, description, optional customer information, reference, expiry, redirect URL and metadata. The system generates a unique URL such as <https://pay.example.com/pay/{token}>.

Checkout must be responsive, show merchant identity, amount and currency, provide payment states, and avoid requiring a customer account. Provider-hosted or tokenized flows should be preferred so the gateway does not unnecessarily handle raw card data.

9. Wallet & Ledger

Each merchant receives a wallet. The wallet should expose available balance and, where relevant, pending balance. Financial movements must be represented through an immutable ledger rather than relying solely on a mutable balance field.

Payment success creates a CREDIT ledger entry. A withdrawal creates a DEBIT/reservation entry. Refunds create reversal entries. Every entry should retain reference, amount, currency, balance before/after, transaction relationship and timestamp.

10. Transaction History

Merchants can view transaction ID, merchant reference, amount, currency, customer, provider, status, payment method and date. Filters should include date range, status, amount, currency, provider, reference and customer email. Search should support transaction ID, reference and email.

11. Balance History

Merchants can view chronological wallet movements, including payments, withdrawals, fees, refunds, reversals and manual adjustments, with resulting balance after each entry.

12. Withdrawals

Merchants can withdraw available funds to a verified bank account. Flow: select bank account → enter amount → validate balance → create withdrawal → reserve/debit funds → process payout → update status.

Withdrawal states: REQUESTED, PROCESSING, SUCCESS, FAILED, CANCELLED.

13. Webhooks

The gateway should provide merchant webhooks for payment and withdrawal events. Example events include payment.pending, payment.success, payment.failed, payment.refunded, withdrawal.pending, withdrawal.success and withdrawal.failed.

Webhook delivery must support signed payloads, retries with exponential backoff, delivery logs and idempotent event handling. Provider webhooks should be isolated from merchant-facing webhooks.

14. Idempotency

Payment creation must support an Idempotency-Key. Repeating a request with the same valid key must not create duplicate payments or duplicate wallet credits.

15. Refunds

Refunds can be introduced in MVP+ or Phase 2. Refund processing must create a corresponding ledger reversal and maintain a complete transaction audit trail.

16. Merchant Settings

Business settings: business name, logo, website, email and phone. Checkout settings: logo, brand name, success URL and cancel URL. Developer settings: webhook URL and webhook secret. Security settings: password, API keys, sessions and future 2FA.

17. Admin Dashboard

Administrators should be able to view and manage merchants, transactions, withdrawals, providers and audit logs. Merchant controls should include view, activate, suspend and inspect financial activity. Provider controls should include configuration, enable/disable status and health monitoring.

18. Suggested API Surface

Area	Endpoints
Authentication	POST /auth/register, POST /auth/login, POST /auth/logout, password reset
API Keys	GET /api-keys, POST /api-keys, DELETE /api-keys/{id}, POST /api-keys/{id}/rotate
Payments	POST /v1/payments, GET /v1/payments, GET /v1/payments/{id}, GET /v1/payments/{id}/verify
Payment Links	POST /payment-links, GET /payment-links, GET /payment-links/{id}, PATCH /payment-links/{id}
Wallet	GET /wallet, GET /wallet/balance, GET /wallet/ledger
Withdrawals	POST /withdrawals, GET /withdrawals, GET /withdrawals/{id}
Provider Webhooks	POST /webhooks/{provider}

19. Data Model

transactions: id, merchant_id, provider_id, customer_id, reference, provider_reference, amount, currency, fee, net_amount, status, payment_method, metadata, checkout_url, timestamps.

wallets: id, merchant_id, currency, available_balance, pending_balance, timestamps.

ledger_entries: id, wallet_id, transaction_id, type, direction, amount, currency, balance_before, balance_after, description, reference, timestamp.

payment_links: id, merchant_id, reference, amount, currency, description, customer information, checkout token, status, expiry, payment_id, metadata, timestamps.

withdrawals: id, merchant_id, wallet_id, amount, fee, currency, bank details, provider, provider_reference, status, failure_reason, timestamps.

20. Security Requirements

Use HTTPS, strong password hashing, encrypted secrets, secure API-key handling, rate limiting, request validation, CSRF protection where applicable, RBAC, audit logs, webhook signature verification, idempotency, encryption of sensitive data and secure secret management. Never log passwords, API secret keys, CVV or raw card data.

21. Reliability & Reconciliation

Target platform availability: $\geq 99.9\%$. Use queues/background workers, database transactions, retries, provider timeout handling, monitoring and alerting. Reconciliation should compare gateway transactions, provider transactions, merchant ledger and provider settlement to identify discrepancies, duplicates or missing records.

22. Recommended Technical Architecture

For the MVP, a modular monolith is recommended rather than immediately splitting into many microservices. Suggested stack: Laravel/PHP backend, PostgreSQL, Redis/queues, Next.js/React dashboard, Docker, Terraform and AWS. Observability can use OpenTelemetry, CloudWatch and Sentry.

Logical components: API Layer → Authentication → Payment Service → Provider Service → Wallet/Ledger → Withdrawal Service → Notifications/Webhooks → PostgreSQL/Redis.

23. MVP Delivery Phases

Phase	Scope
1 — Foundation	Registration, login, dashboard, authentication, API keys, merchant settings
2 — Payments	Payment API, Cray integration, initialization, verification, webhooks, transaction history
3 — Checkout	Payment links, hosted checkout, success/failure pages, merchant branding
4 — Wallet	Wallet, ledger, available/pending balances, balance history
5 — Withdrawals	Bank accounts, withdrawal requests, status/history, provider payout
6 — Admin	Merchant, transaction, withdrawal, provider and audit management

24. MVP Acceptance Criteria

A merchant can register, log in, generate API keys, initialize a payment, generate/share a payment link, receive a successful customer payment, see the transaction in the dashboard, see the corresponding wallet credit, view balance/ledger history and initiate a withdrawal.

A customer can open a payment link, view merchant/payment details, complete payment without creating an account, and receive a success/failure result.

A developer can initialize payments, receive a checkout URL, receive signed webhooks, verify transactions and use their own merchant reference.

25. Key Business Rules

1. Only independently verified successful payments can credit a merchant wallet.
2. Duplicate provider webhooks must never create duplicate ledger credits.
3. A merchant cannot withdraw more than available balance.
4. Failed payments must never credit the wallet.
5. Refunds and reversals must create corresponding ledger entries.
6. The ledger is the financial source of truth; displayed balances may be cached/derived values.

26. Future Roadmap

V2: multiple payment providers, routing, automatic failover, refunds, recurring payments, customer records, saved payment methods, multiple currencies and team roles. V3: split payments, marketplace sub-accounts, multi-country support, advanced reconciliation, fraud detection, disputes and FX. V4: payment orchestration, intelligent provider routing, provider cost optimization, smart retries and cross-border payments.

27. Success Metrics

Metric	Target
Platform availability	≥99.9%
Checkout availability	≥99.9%
Webhook delivery	≥99.9%
Duplicate wallet credits	0
Unreconciled transactions	0
API response time	<500ms excluding provider latency
Checkout page load	<2.5 seconds
Payment verification accuracy	100%
Withdrawal processing accuracy	100%

28. Product Principle

The core of the product should be the payment abstraction layer and the financial ledger—not merely the checkout UI. Merchants should integrate with the gateway's API once, while the platform handles provider selection, normalization, verification, wallet accounting and settlement workflows behind the scenes.

Target end state: Merchant → Your Gateway API → Payment Router → Cray / Future Providers → Verification → Merchant Ledger → Wallet → Withdrawal.

End of PRD